EE-170L Computer Programming Lab

Relational Operators / logical operators/Conditionals

1 Objective

The students will learn the following topics during this lab session.

1. Comparison/Relational Operators
2. Logical Operators
3. **if** statement
4. **if else** statement
5. **else if** statement

2 Relational Operators in C++

C++ Symbol	Math Notation	Meaning
<	$<$	Less than
>	$>$	Greater than
<=	$\leq$	Less than or equal to
>=	$\geq$	Greater than or equal to
==	$=$	Equal to
! =	$\neq$	Not equal to

```
#include <iostream>
using namespace std;

int main() {
    int age;

    cout << "Enter your age: ";
    cin >> age;

    if (age >= 18) {
        cout << "You are eligible to vote." << endl;
    }
    else {
        cout << "You are not eligible to vote." << endl;
    }

    return 0;
}
```

3 Logical Operators in C++

C++ Symbol	Syntax	Example
!	Logical negation (NOT)	!*a*
&&	Logical AND	*a* && *b*
||	Logical OR	*a* || *b*

```cpp
#include <iostream>
using namespace std;

int main() {
    int age;
    char hasID;

    cout << "Enter your age: ";
    cin >> age;

    cout << "Do you have an ID card? (y/n): ";
    cin >> hasID;

    // Using logical AND (&&)
    if (age >= 18 && hasID == 'y') {
        cout << "You can enter." << endl;
    }
    else {
        cout << "Entry not allowed." << endl;
    }

    return 0;
}
```

4 Decision Control Structure in C++

In C++ programming the instructions are executed sequentially, by default. At times, we need a set of instructions to be executed Depending upon different conditions to be satisfied. In such cases we have to use decision control instructions. Here we discuss the following ways for decision making in C++.

1. The **if** statement
2. The **if-else** statement
3. The nested **else-if** statement

4.1 The if statement

The if statement controls conditional branching. The body of an **if** statement is executed if the value of the expression/condition specified in the if statement is true. The syntax of the if statement is as follows:

Syntax:

If (expression)

{

Block of statement;

}

Example:

Write a program in which it takes a number from keyboard as an input and if the number is greater than 100 it prints “The number is greater than hundred”.

```
#include <iostream>
using namespace std;

int main() {
    int number;

    cout << "Enter a number: ";
    cin >> number;

    if (number > 100) {
        cout << "The number is greater than hundred" << endl;
    }

    return 0;
}
```

4.2 The if-else statement

if - else statement is similar to **if** with the addition of **else** statement. If the condition is false in **if** then its body will be skipped and the **else** statement’s body will be executed.

Syntax:

If (expression)

{

Block of statement;

}

else

{

Block of statement;

}

Example:

Write a program in which it takes two numbers from keyboard as input and subtract larger number from smaller.

```
#include <iostream>
using namespace std;
int main()
{
    int a, b;
    cout << "Enter first integer number : \n";
    cin >> a;
    cout << "Enter second integer number : \n";
    cin >> b;
    if (a > b)
        cout << a << " - " << b << " = " << a - b;
    else
        cout << b << " - " << a << " = " << b - a;
    return 0;
}
```

LabCodes/lab04p02.cpp

4.3 else-if statement

Sometimes we wish to make a multi-way decision based on several conditions. The most general way of doing this is by using the else if variant on the if statement. This works by cascading several comparisons. As soon as one of these gives a true result, the following statement or block is executed, and no further comparisons are performed.

Syntax:

```
if(expression)
{
Block of statement;
}
else if(expression)
{
Block of statement;
}
else if(expression)
{
Block of statement;
}
... // you can add as many else-if statements as many you need
...
...
else
{
Block of statement;
}
```

Lab03: Lab Tasks

1. Comparison/Relational Operators

   Write a C program that prompts the user to enter two numbers.

   Use comparison operators to check if the first number is equal to, greater than, or less than the second number.

   Display appropriate messages based on the comparison results.

2. Logical Operators

   Write a C program that prompts the user to enter an integer.

   Use logical operators to check if the entered number is greater than 0 and less than 100.

   Display a message indicating whether the number meets both conditions or not.

3. If Statement

   Write a C program that prompts the user to enter an integer.

   Use an if statement to check if the entered number is positive.

   Display a message indicating whether the number is positive or not.

4. If-else Statement

   Write a C program that prompts the user to enter their age.

   Use an if-else statement to check if the entered age is greater than or equal to 18.

   Display appropriate messages for both conditions.

5. Else-if Statement

   Write a C program that prompts the user to enter an integer.

   Use an else-if statement to check if the entered number is positive, negative, or zero.

   Display appropriate messages based on the entered number.

6. A separate task sheet will be provided for more practice.